

Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Tecnicatura Universitaria en Inteligencia Artificial

Trabajo Práctico N° 1

Redes neuronales feed forward
y convolucionales

Aprendizaje Automático II

Grupo 5

Integrantes	Legajo
Facundo Molina	M-7128/5
Martín Facundo Lapolla	L-3372/3
Jonatan Gallardo	G-5970/6
Gabriel Soda	S-5679/1

Rosario, octubre de 2026

Introducción y criterio de trabajo

El trabajo aplica redes neuronales a problemas de clasificación y regresión. Este informe resume el procedimiento, las decisiones, los desafíos y los aprendizajes. El código, los gráficos y el detalle de los experimentos se encuentran en los notebooks de cada ejercicio.

Usamos la semilla 42 en los ejercicios 1 y 2, y la 31 en los ejercicios 3 y 4. Separamos entrenamiento, validación y test. Train ajusta los pesos y validación permite monitorear el entrenamiento. Test permite reportar el desempeño final. Los ejercicios 3 y 4 también incluyen comparaciones de varios modelos sobre test, una limitación que se señala en sus conclusiones. Los escaladores y codificadores se ajustan con train. En el ejercicio 3, la agrupación de estados poco frecuentes se definió antes de la partición.

1. Clasificación binaria de ACV

1.1. Procedimiento y datos

Trabajamos con 5.110 pacientes. La variable objetivo es `stroke`. Hay 249 positivos (4,87 %) y 4.861 negativos (95,13 %). No se encontraron filas duplicadas ni identificadores repetidos. Los únicos faltantes fueron 201 valores de BMI (3,93 %).

Exploramos las variables con histogramas, boxplots, correlaciones y proporciones por categoría. Edad presentó la mayor correlación de Spearman con `stroke` entre las variables examinadas (0,25). Glucosa y BMI mostraron valores extremos y solapamiento entre clases. Conservamos esos extremos para no eliminar posibles casos relevantes. Estas asociaciones no prueban causalidad.

Eliminamos `id` y dividimos de forma estratificada en 3.066 pacientes para train, 1.022 para validación y 1.022 para test. Validación y test tienen 50 positivos cada uno.

1.2. Decisiones y justificación

Completamos BMI con la mediana de train por intervalos de edad de 20 años y género. La mediana global de train funciona como respaldo. Elegimos medianas porque los valores extremos influyen menos que sobre la media. La agrupación permite incorporar diferencias entre grupos, aunque no se probó que supere otras imputaciones.

Usamos One-Hot para categorías y variables binarias, sin asignarles un orden artificial. Dejamos una categoría de referencia con `drop="first"`. Conservamos `Unknown` en tabaquismo porque es una categoría explícita. Estandarizamos edad, glucosa y BMI con `StandardScaler`. El resultado tiene 16 entradas.

El MLP de referencia tiene arquitectura 16-64-1 y 1.153 parámetros. Usa ReLU y dropout de 0,3 en la capa oculta. La salida entrega un logit y la pérdida es `BCEWithLogitsLoss`. Aplicamos sigmoid al evaluar para obtener scores entre 0 y 1.

Entrenamos con Adam, tasa 0,001, `weight_decay=0,0001` y batches de 64. Permitimos hasta 500 épocas, con paciencia 30 y mejora mínima de pérdida de validación de 0,0001. Restauramos los mejores pesos para limitar el sobreajuste.

1.3. Experimentos y resultados

Las curvas y la evaluación en test se muestran en el Anexo A.

Comparamos BorderlineSMOTE, aplicado solo a train, con la ponderación de positivos en la pérdida. BorderlineSMOTE llevó ambas clases a 2.917 registros. La ponderación mantuvo los pacientes originales y usó `pos_weight=2917/149`, aproximadamente 19,58. También comparamos tres arquitecturas con ponderación.

Arquitectura	Parámetros	F2 en validación	Recall en validación
16-64-1	1.153	0,4366	84 %
16-32-1	577	0,4343	82 %
16-64-32-1	3.201	0,4310	80 %

Elegimos 16-64-1 por su mayor F2. Las diferencias fueron pequeñas y corresponden a una sola semilla. El modelo de 32 neuronas sigue siendo una alternativa razonable por su menor tamaño.

Seleccionamos los umbrales con F2 en validación. Esta métrica da más importancia a recall que F1, porque buscamos reducir los falsos negativos. Con el umbral elegido para cada método, `pos_weight` obtuvo F2 de 0,4366 frente a 0,3779 de BorderlineSMOTE. Detectó 42 positivos frente a 39 y generó 239 falsos positivos frente a 277.

La configuración final fue el modelo ponderado, con pesos de la época 16 y umbral 0,567368. El entrenamiento se detuvo en la época 46. La pérdida de train seguía bajando cuando validación dejó de mejorar, lo que respaldó recuperar un checkpoint anterior. Las pérdidas ponderadas no se comparan directamente con las de BCE sin ponderar.

Métrica de test	Resultado	Métrica de test	Resultado
Accuracy	75,64 %	Precisión	14,34 %
Recall	80,00 %	F1	0,2432
F2	0,4175	AUC-ROC	0,8487
Especificidad	75,41 %	Precisión promedio (AP)	0,3151

La matriz de test contiene 733 verdaderos negativos, 239 falsos positivos, 10 falsos negativos y 40 verdaderos positivos. Una predicción siempre negativa tendría 95,11 % de accuracy, pero no detectaría ningún positivo. Por eso accuracy sola resulta engañosa en este problema.

1.4. Desafíos, limitaciones y aprendizajes

El principal desafío fue detectar positivos sin producir demasiadas falsas alertas. Detectamos 40 de 50 casos, pero 239 de las 279 alertas fueron incorrectas. Los 10 falsos negativos podrían quedar sin señalar si se usara el resultado para descartar una revisión. El modelo no fue validado para tomar decisiones clínicas.

Aprendimos que tratar el desbalance y elegir el umbral son decisiones distintas. F2 permitió explicitar la prioridad de recall, pero no impuso una precisión mínima aceptable. Las limitaciones principales son los pocos positivos, una sola partición y varias decisiones sobre la misma validación. BorderlineSMOTE también puede producir indicadores categóricos fraccionarios tras One-Hot. Como mejoras proponemos repetir particiones, comparar una regresión logística ponderada y probar imputaciones o sobremuestreo adecuados para datos mixtos.

2. Clasificación multiclase de dígitos

2.1. Procedimiento y datos

Usamos MNIST, con imágenes de dígitos del 0 al 9 en escala de grises y tamaño 28×28 . Leímos los cuatro archivos IDX y verificamos encabezados, cantidades, dimensiones y etiquetas. Dividimos las 60.000 imágenes originales de train en 48.000 para entrenamiento y 12.000 para validación mediante estratificación. Conservamos las 10.000 imágenes oficiales de test.

Mostramos una grilla de imágenes, las frecuencias de cada dígito y los histogramas de intensidad. Las clases de train representan entre 9,04 % y 11,24 % del total. Consideramos el conjunto aproximadamente balanceado y no aplicamos sobremuestreo. En los histogramas predominan los píxeles negros del fondo.

2.2. Decisiones y justificación

Aplanamos cada imagen en 784 valores y convertimos los píxeles a float32. Dividimos por 255 para llevarlos a $[0, 1]$. Esta transformación fija conserva el orden y las frecuencias de las intensidades. Usamos intervalos equivalentes en ambos histogramas para comparar correctamente el antes y el después.

Mantenemos las etiquetas como enteros de 0 a 9 y las convertimos a `torch.long`. CrossEntropyLoss acepta esos índices, por lo que no hace falta usar one-hot. El modelo entrega logits y no aplica softmax antes de la pérdida. Softmax se usa para analizar la confianza.

Componente	Configuración y motivo
Arquitectura	784-128-64-10, con 109.386 parámetros. Dos capas ocultas para combinar patrones con un tamaño moderado.
Activaciones	ReLU en las capas ocultas para aprender relaciones no lineales. Salida lineal con diez logits.
Regularización	Dropout de 0,3 y decaimiento de pesos de 0,0001 para limitar el sobreajuste.
Optimización	AdamW con tasa inicial de 0,001 y batches de 64.
Tasa de aprendizaje	ReduceLROnPlateau, factor 0,5 y paciencia 2, según pérdida de validación.
Selección de pesos	Hasta 30 épocas y early stopping tras 7 épocas sin nueva pérdida mínima de validación.

Al terminar cada época medimos train y validación con pesos fijos y dropout desactivado. También conservamos las métricas obtenidas durante las actualizaciones. Así distinguimos el proceso de optimización de la comparación entre conjuntos.

2.3. Entrenamiento y sobreajuste

El Anexo B reúne las curvas, la matriz y los ejemplos de errores.

La menor pérdida de validación se obtuvo en la época 18 (0,0793). El entrenamiento se detuvo en la 25 y recuperamos esos pesos. Entre las épocas 18 y 25, la pérdida de train en evaluación bajó de 0,0143 a 0,0082. La de validación osciló y terminó en 0,0834.

En la última época, train en evaluación alcanzó aproximadamente 99,8 % de accuracy frente a 98,0 % de validación. Esta diferencia muestra sobreajuste, aunque no hubo una caída sostenida fuerte de accuracy de validación. El cruce con la curva de train durante las actualizaciones no basta para identificar su inicio, porque esa curva incluye dropout activo.

2.4. Resultados y análisis de errores

El checkpoint seleccionado obtuvo 97,94 % de accuracy y pérdida 0,0756 en test. Acertó 9.794 imágenes y falló en 206. Accuracy es un resumen adecuado por el balance aproximado de las clases, pero la complementamos con resultados por dígito y la matriz de confusión.

Dígito	Aciertos por clase	Dígito	Aciertos por clase
0	99,08 %	5	97,76 %
1	99,12 %	6	98,23 %
2	97,97 %	7	97,57 %
3	98,22 %	8	96,61 %
4	98,17 %	9	96,53 %

Los aciertos por clase se calculan sobre los ejemplos reales de cada dígito, por lo que corresponden a su recall. Las confusiones más frecuentes fueron 9 hacia 4 (13), 7 hacia 2 (8), 4 hacia 9 (7) y 7 hacia 9 (7). Agrupando ambas direcciones, 4 y 9 acumularon 20 errores. Los lazos abiertos o cerrados y los trazos diagonales ofrecen posibles explicaciones visuales.

Analizamos los cinco errores de mayor confianza, mostrados en el notebook. Sus índices de test son 3520, 1681, 2654, 4601 y 5331. Corresponden a 6 hacia 4, 3 hacia 7, 6 hacia 1, 8 hacia 4 y 1 hacia 6. Todos superaron 99,95 % de confianza softmax. Algunos tienen trazos ambiguos, pero el último no ofrece una semejanza clara con un 6. Estas observaciones no prueban qué características usó el modelo ni que las etiquetas estén mal.

2.5. Desafíos, limitaciones y aprendizajes

Los desafíos fueron reconocer distintas formas de escritura y comparar correctamente las curvas. Medir train y validación sin dropout permitió observar una brecha que era menos evidente con las métricas tomadas durante las actualizaciones.

El MLP conserva los píxeles al aplanar, pero debe aprender las relaciones entre sus posiciones. Una CNN podría aprovechar filtros locales y compartirlos en distintas partes de la imagen para detectar bordes, curvas y uniones. No se implementó una CNN en este ejercicio ni se demostró que mejore el resultado.

Aprendimos que una buena accuracy no garantiza confianza fiable. El resultado tampoco asegura rendimiento sobre fotos, fondos distintos o varios dígitos en una imagen. Como mejoras futuras proponemos comparar arquitecturas y pequeñas transformaciones solo sobre train, cuidando que no cambien la identidad del dígito.

3. Regresión de ganancias

3.1. Procedimiento y datos

Trabajamos con 9.994 registros y 21 columnas del dataset Superstore. La variable objetivo es `Profit`, expresada en dólares. Puede tomar valores positivos y negativos. No se detectaron faltantes. Su mediana es 8,67 USD y su media es 28,66 USD. El criterio de 1,5 veces el rango intercuartílico identifica 1.881 valores atípicos, equivalentes al 18,82 %. Conservamos esos registros porque un valor extremo no implica por sí mismo un error de carga. El Anexo C muestra su distribución.

El análisis de correlación y los gráficos de dispersión muestran que Sales se asocia positivamente con Profit, mientras que Discount se asocia negativamente. Las correlaciones de Spearman son aproximadamente 0,52 y -0,52. También observamos mayor dispersión de ganancias y pérdidas en operaciones de mayor venta. Estas asociaciones no prueban causalidad.

Eliminamos identificadores, nombres y variables redundantes o de alta cardinalidad, como City, Postal Code y Product Name. Agrupamos en Other los estados con menos de 30 registros. Esta agrupación se calculó sobre el dataset completo antes del split, por lo que utiliza frecuencias de validación y test. Es una limitación del procedimiento. El escalado y One-Hot sí se ajustaron exclusivamente con train.

De Order Date extraímos año y trimestre como categorías, y mes y día de la semana mediante seno y coseno. Calculamos los días entre la orden y el envío. El split aleatorio dejó 6.995 registros en train, 999 en validación y 2.000 en test. No es una evaluación temporal de ventas futuras.

3.2. Decisiones y justificación

Aplicamos RobustScaler a Sales, Quantity, Discount y los días hasta el envío. Usa mediana y rango intercuartílico, lo que reduce la influencia de los extremos sobre los parámetros del escalado. Las variables cíclicas pasan sin cambios y las categóricas se codifican con One-Hot, ignorando categorías nuevas al transformar. La entrada final tiene 86 características.

También escalamos Profit con un RobustScaler ajustado solo en train. Las pérdidas se calculan en esa escala transformada y las predicciones se revierten a dólares para informar MAE y RMSE. En Huber, el umbral $\delta=1$ corresponde a una unidad de Profit escalado, no a un dólar.

El MLP tiene arquitectura 86-128-64-32-1 y 21.505 parámetros entrenables. Usa ReLU y dropout de 0,2 en las capas ocultas. La salida lineal permite predecir tanto ganancias como pérdidas. Entrenamos con Adam, tasa inicial de 0,001, batches de 64 y weight decay de 0,0001. Permitimos hasta 300 épocas, con early stopping de paciencia 25 y restauración de los mejores pesos. ReduceLROnPlateau reduce la tasa a la mitad después de 10 épocas sin mejora.

Comparamos MSE, MAE y Huber con la misma arquitectura, inicialización y orden de batches. MSE penaliza más los errores grandes. MAE crece linealmente con el error absoluto. Huber combina una región cuadrática para errores pequeños con una región lineal para errores grandes. Estas propiedades justifican la comparación, pero no garantizan de antemano qué modelo rendirá mejor.

3.3. Experimentos y resultados

Pérdida	Mejor época	Épocas ejecutadas	MAE val. (USD)	RMSE val. (USD)
MSE	98	123	20,397	49,451
MAE	50	75	15,966	41,842
Huber ($\delta=1$)	49	74	16,379	41,738

Huber obtuvo el menor RMSE de validación, aunque la diferencia frente a MAE fue de solo 0,104 USD. MAE obtuvo el menor error absoluto medio. La elección de Huber es compatible con priorizar RMSE en validación y buscar un compromiso entre errores pequeños y grandes. No demuestra que sea superior en todas las métricas ni en otras particiones.

En las curvas de Huber, train y validación bajan al principio. Luego de la mejor época, la validación deja de mejorar mientras train sigue bajando. Esto indica una brecha de generalización y respalda restaurar los pesos de la época 49. La pérdida de train se registra durante las actualizaciones, con dropout activo y pesos cambiantes. Por eso no es directamente equivalente a evaluar todo train con los pesos finales de cada época.

El notebook también evalúa los tres modelos sobre test y guarda esta comparación:

Función de costo	MAE test (USD)	RMSE test (USD)	R ² test
MSE	25,30	161,75	0,7242
Huber ($\delta=1$)	24,16	166,21	0,7087
MAE	24,48	189,10	0,6230

Para Huber, el error absoluto medio fue de 24,16 USD por registro. No significa que todos los errores sean cercanos a ese valor. El RMSE de 166,21 USD refleja el mayor peso de unos pocos errores grandes. El R² de 0,7087 representa una reducción de aproximadamente 71 % del error cuadrático respecto de predecir la media del Profit de test. MSE obtuvo menor RMSE en este test, aunque eso no estaba garantizado por su función de pérdida.

3.4. Residuos, desafíos y aprendizajes

El gráfico de residuos usa $y_{\text{real}} - y_{\text{pred}}$ frente al valor real, como pide la consigna. Los mayores errores se concentran en operaciones extremas. Para Huber, los cinco errores absolutos más grandes fueron:

Posición en test	Profit real (USD)	Predicción (USD)	Error absoluto (USD)
44	-1.811,08	3.134,41	4.945,49
273	-6.599,98	-2.995,87	3.604,10
1727	-3.839,99	-861,82	2.978,17
977	8.399,98	6.188,93	2.211,05
960	5.039,99	4.541,29	498,70

En el caso 44, el modelo predice una ganancia cuando hubo pérdida. En los casos 273 y 1727, subestima la magnitud de la pérdida. En 977 y 960, subestima la ganancia. Son posiciones de la tabla de test, no identificadores originales de la transacción. Estos casos muestran por qué un MAE bajo no basta para estimar el riesgo de las operaciones grandes.

Los principales desafíos fueron conservar los extremos y manejar las categorías. Aprendimos a comparar pérdidas mediante métricas comunes en dólares. Consultar varios modelos en test limita su independencia como evaluación final. Además, una sola partición no permite asegurar estabilidad. Como mejoras proponemos aprender la agrupación de estados solo con train, validar por tiempo o por pedido según el uso previsto, y comparar modelos de árboles con el mismo protocolo. No medimos esa comparación, por lo que no podemos afirmar que sean más precisos o más rápidos.

4. Clasificación de razas con CNN

4.1. Procedimiento y datos

Trabajamos con diez clases del dataset de imágenes de perros: Beagle, Bulldog, Cockapoo, Coyote, Dingo, Doberman, Labradoodle, Poodle, Pug y Rottweiler. La selección combina grupos visualmente parecidos para analizar sus confusiones. Conservamos las particiones provistas, con 1.181 imágenes de train, 100 de validación y 100 de test. Validación y test tienen diez imágenes por clase.

Las imágenes se reescalan a 256 píxeles en su lado menor y se recortan en el centro a 224×224 . Luego se convierten a tensores y normalizan con las medias y desviaciones estándar de ImageNet. Se usan esos valores fijos de normalización, sin cargar redes ni pesos preentrenados. Las CNN se construyen y entrenan desde cero.

Train presenta un desbalance moderado, desde 83 imágenes de Coyote hasta 148 de Pug. En E4 se agregan 299 referencias a imágenes de train con transformaciones aleatorias para obtener 1.480 muestras por época, 148 por clase. Los aumentos incluyen recortes, giros horizontales, rotaciones leves y cambios de color. Se transforman las referencias añadidas, mientras las originales mantienen el recorte central. Validación y test no reciben aumentos.

4.2. Evolución de arquitecturas y entrenamiento

E1 usa tres bloques Conv-BatchNorm-ReLU-MaxPool y una cabeza densa. E2 aumenta a seis convoluciones, incorpora un filtro inicial de 5×5 y modifica el submuestreo con stride. E3 agrega un módulo Inception con ramas paralelas y dos bloques residuales. E4 conserva la estructura de E3 y añade dropout de 0,5 en la cabeza, weight decay de 0,0001 y el entrenamiento balanceado con aumentos.

Las cuatro redes incluyen BatchNorm y early stopping. E4 incorpora además dropout, weight decay y aumento de datos sobre esa base.

Usamos CrossEntropyLoss sobre logits, Adam con tasa inicial de 0,001 y batches de 32. ReduceLROnPlateau reduce la tasa a la mitad con paciencia 3. El límite es de 100 épocas, con early stopping de paciencia 10 según pérdida de validación. El bucle restaura los mejores pesos y guarda checkpoints con estados del modelo, optimizador, scheduler e historial para permitir la reanudación.

4.3. Comparación de experimentos

Las accuracies de train y validación corresponden a la evaluación posterior con los pesos recuperados. El helper del notebook redondea las proporciones a dos decimales antes de convertirlas a porcentaje. Por eso las cifras de train se presentan como aproximaciones.

Modelo	Train aprox.	Validación	Test	Parámetros	Épocas	Mejor época
E1	99 %	85 %	67 %	620.586	34	24
E2	97 %	79 %	68 %	1.626.538	35	25
E3	97 %	77 %	62 %	2.318.474	24	14
E4	96 %	83 %	76 %	2.318.474	53	43

Épocas indica la duración total del entrenamiento. Mejor época identifica los pesos recuperados por menor pérdida de validación.

E2 aumentó la capacidad con una mejora pequeña en test frente a E1. E3 agregó bloques avanzados, pero su resultado en test fue menor. E4 obtuvo el mayor porcentaje de aciertos en test dentro de estas cuatro ejecuciones. Como cambió varias técnicas a la vez, no podemos atribuir esa diferencia a una sola de ellas. Una única semilla y 100 imágenes de test tampoco bastan para asegurar una ventaja estable.

4.4. Curvas y análisis de errores de E4

El desarrollo destaca E4 y presenta sus curvas y errores en el Anexo D. Se entrenó durante 53 épocas en aproximadamente 150 minutos sobre CPU. Su menor pérdida de validación fue 0,5567 en la época 43. Después, validación dejó de mejorar mientras train continuó bajando. La regularización moderó el sobreajuste, pero no lo eliminó. La evaluación de los pesos recuperados muestra una brecha aproximada de 13 puntos entre train y validación.

Las curvas de train registran las actualizaciones con dropout y aumentos activos. Validación usa modo evaluación y datos sin aumentos. Esto ayuda a explicar que validación pueda superar a train durante parte del entrenamiento. El cruce entre ambas curvas no identifica por sí solo el inicio del sobreajuste.

La matriz de E4 suma 76 aciertos y 24 errores en 100 imágenes. Beagle y Labradoodle alcanzan recall de 90 %. Poodle y Pug tienen el menor recall, con 60 %. Labradoodle tiene precisión de aproximadamente 56 %, ya que también recibe predicciones incorrectas de otras clases. Así, una clase puede tener buen recall y menor precisión.

La confusión entre Doberman y Rottweiler suma cinco casos, tres en una dirección y dos en la otra. Dos Poodle se predicen como Cockapoo y otros dos como Labradoodle. El pelaje rizado, los colores y los encuadres ofrecen posibles explicaciones visuales. Las cinco predicciones correctas y las cinco incorrectas del anexo ilustran esas dificultades, pero no prueban qué rasgos internos utilizó la red.

4.5. Limitaciones y aprendizajes

El desarrollo compara el test de todas las arquitecturas y justifica la preferencia por E4 con ese resultado. E1 tuvo mayor accuracy de validación y menor pérdida de validación, 0,5176 frente a 0,5567 de E4. Por eso presentamos E4 como el modelo destacado en el desarrollo, sin afirmar que fue seleccionado exclusivamente por validación. La consulta reiterada de test limita su uso como evaluación independiente final.

Aprendimos que aumentar profundidad o incorporar módulos complejos no garantiza una mejor generalización. La comparación debe controlar tanto la arquitectura como el estado inicial, los datos y el entrenamiento. Como mejoras futuras proponemos separar la selección de la evaluación final, variar una técnica por vez y repetir semillas. El conjunto pequeño de evaluación y las diferencias entre las condiciones de train y validación requieren interpretar las curvas con cuidado.

Conclusión general

Los cuatro ejercicios mostraron que el diseño de una red depende tanto de los datos y la métrica como de sus capas. En ACV, el desbalance y el umbral cambiaron la utilidad de las predicciones. En MNIST, distinguir las métricas de optimización de las de evaluación permitió leer mejor las curvas. En regresión, las métricas en dólares y los residuos mostraron errores que un promedio puede ocultar. En las CNN, la capacidad adicional exigió regularización y una comparación experimental controlada. En todos los casos, las conclusiones quedan acotadas a las particiones y ejecuciones registradas.

Fuentes del desarrollo

Consigna: TP 1 - Redes Feed Forward y Convolucionales.pdf. Código y resultados: AA2_tp1_problema1.ipynb, AA2_tp1_problema2.ipynb, AA2_TP1_E3.ipynb y AA2_TP1_E4.ipynb. Las cifras y figuras provienen de las salidas guardadas en esos notebooks. No se reentrenaron modelos para elaborar este informe.

Anexo A. Imágenes del ejercicio 1

Gráficos de AA2_tp1_problema1.ipynb.

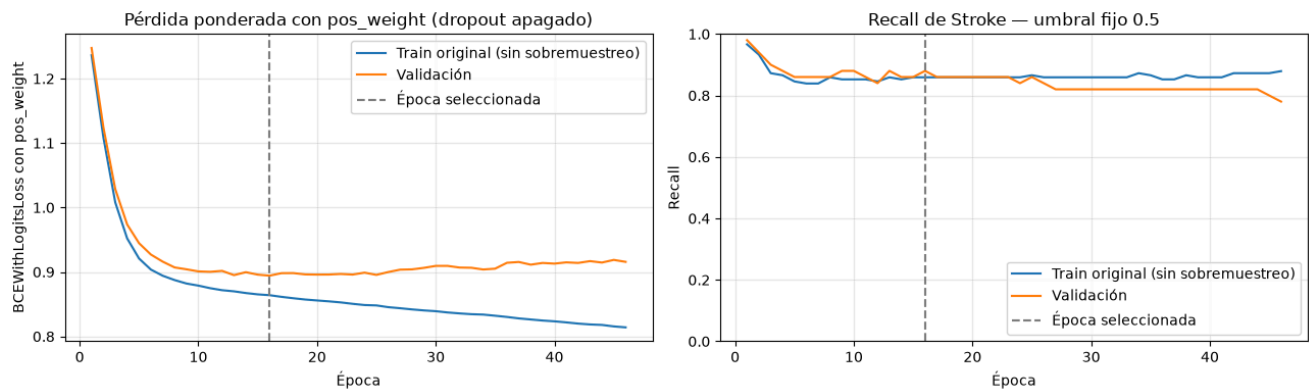


Imagen 1. Modelo con pos_weight. Train y validación se miden sin dropout. La línea señala la época 16. El recall usa umbral 0,5, distinto del umbral final elegido por F2.

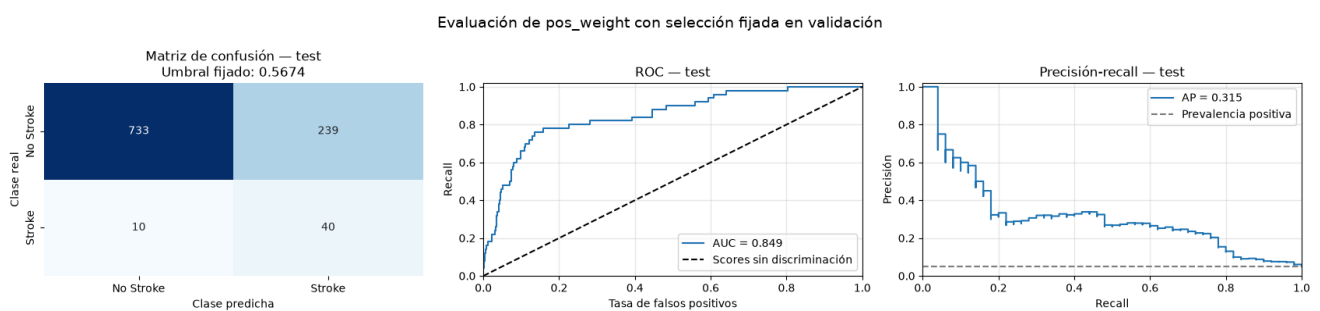


Imagen 2. Test con el umbral fijado en validación. Se detectaron 40 positivos, se omitieron 10 y hubo 239 falsas alertas. ROC y precisión-recall resumen los scores.

Anexo B. Imágenes del ejercicio 2

Gráficos de AA2_tp1_problema2.ipynb.

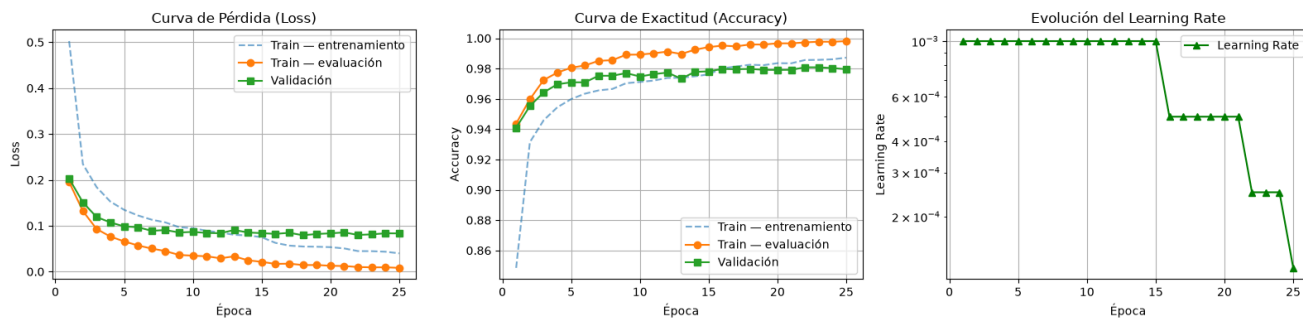


Imagen 3. Curvas de pérdida, accuracy y tasa de aprendizaje. Train en evaluación y validación usan los mismos pesos finales de cada época, sin dropout. La línea discontinua registra las actualizaciones.

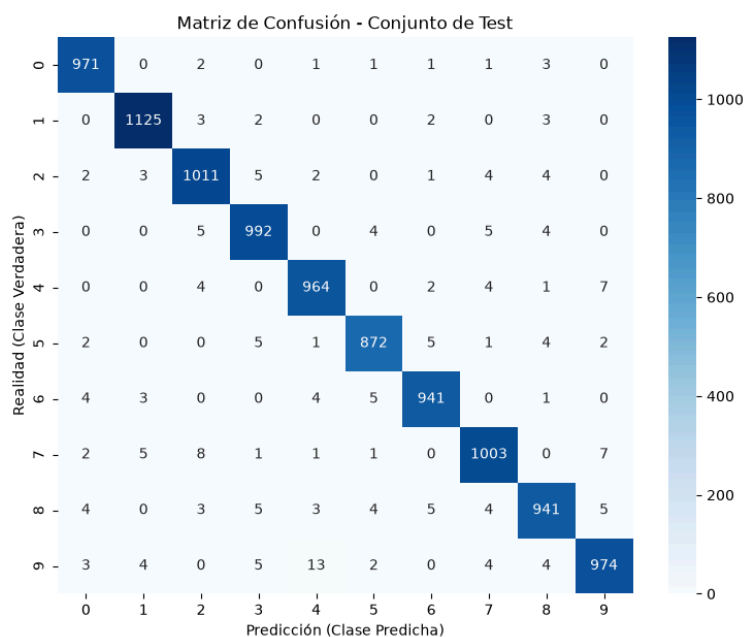


Imagen 4. Matriz de test. Filas: dígito real. Columnas: predicción. Las confusiones entre 4 y 9 suman 20 errores.

Muestras Incorrectas con Mayor Confianza



Imagen 5. Cinco errores de mayor confianza softmax. Todos superan 99,95 %, aunque no coinciden con la etiqueta real.

Anexo C. Imágenes del ejercicio 3

Gráficos de AA2_TP1_E3.ipynb.

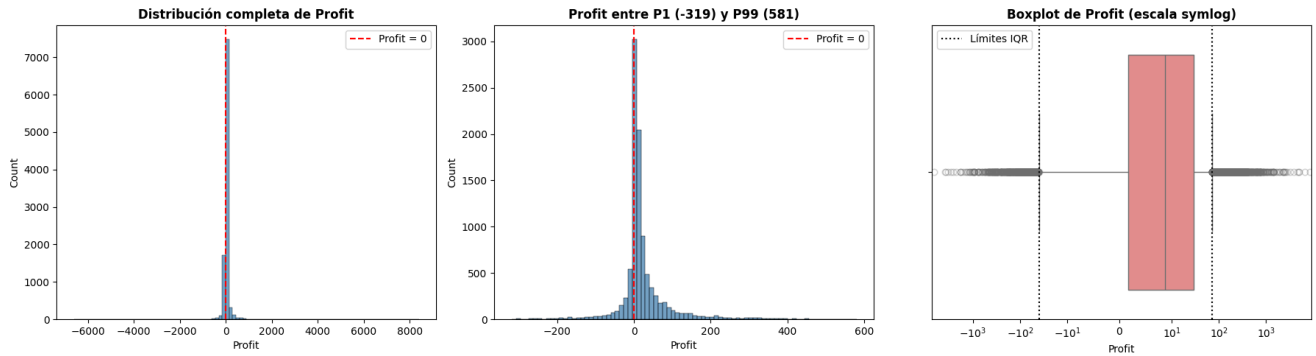


Imagen 6. Distribución de Profit. El panel central acota la visualización entre los percentiles 1 y 99. Los extremos se conservaron en el dataset.

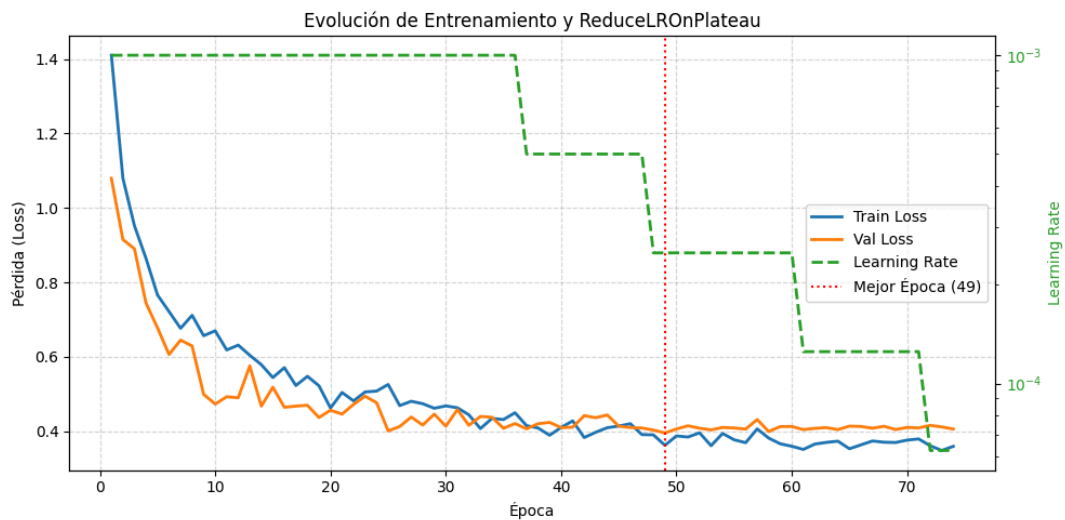


Imagen 7. Pérdidas de Huber en la escala transformada y tasa de aprendizaje. La mejor época fue la 49. Después, validación se estabiliza mientras train continúa bajando.

Error de Predicción vs. Valor Real en Conjunto de Test

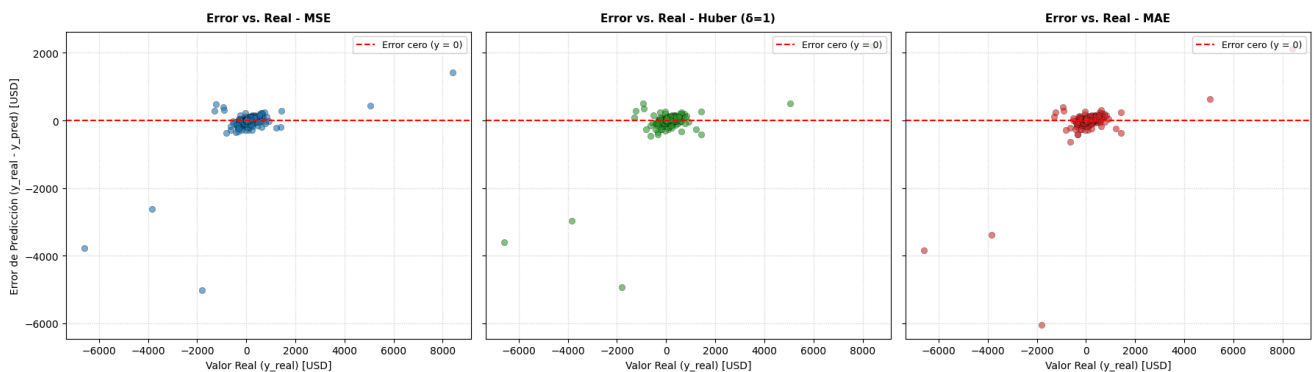


Imagen 8. Residuos frente al Profit real para MSE, Huber y MAE. Los valores positivos indican subestimación del Profit y los negativos sobreestimación. Los mayores errores corresponden a operaciones extremas.

Anexo D. Imágenes del ejercicio 4

Gráficos de la evaluación de E4 guardados en AA2_TP1_E4.ipynb.

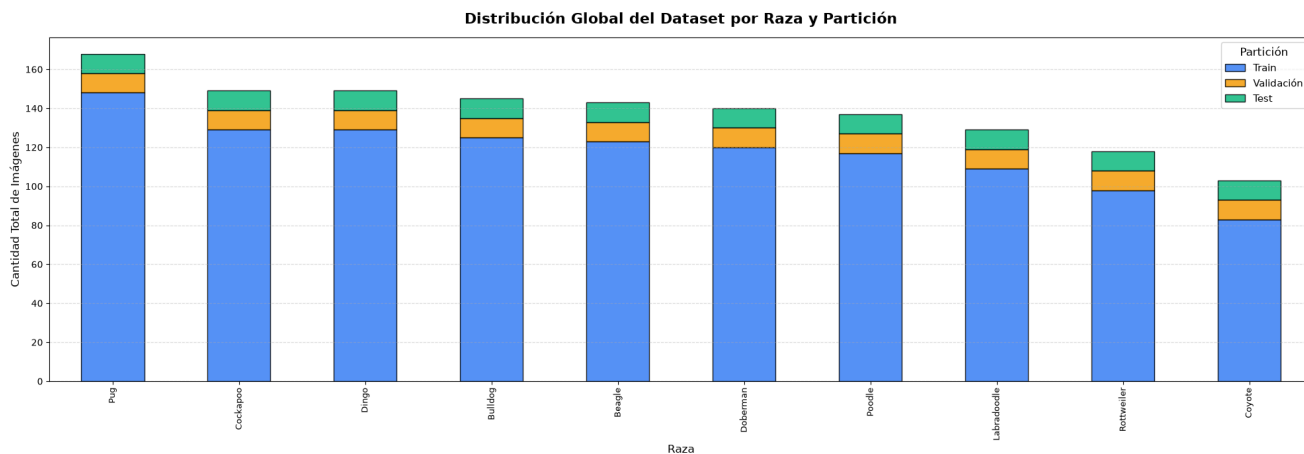


Imagen 9. Distribución por clase y partición. Train tiene entre 83 y 148 imágenes por clase. Validación y test tienen diez por clase.

Curvas de Entrenamiento - E4

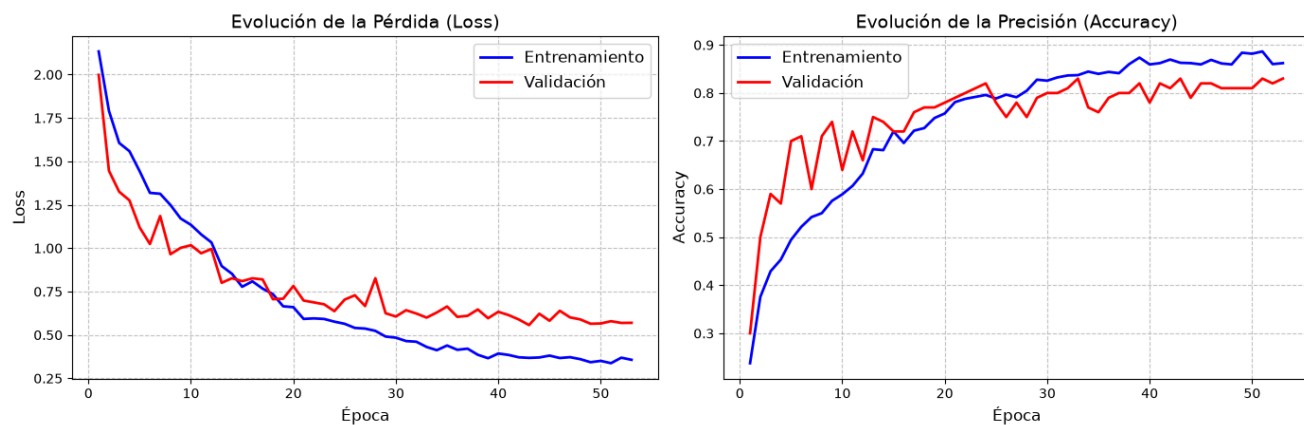


Imagen 10. Curvas de pérdida y accuracy de E4. El registro informa la menor pérdida de validación en la época 43. Las curvas muestran una brecha al final del entrenamiento.

Anexo D. Matriz y ejemplos de E4

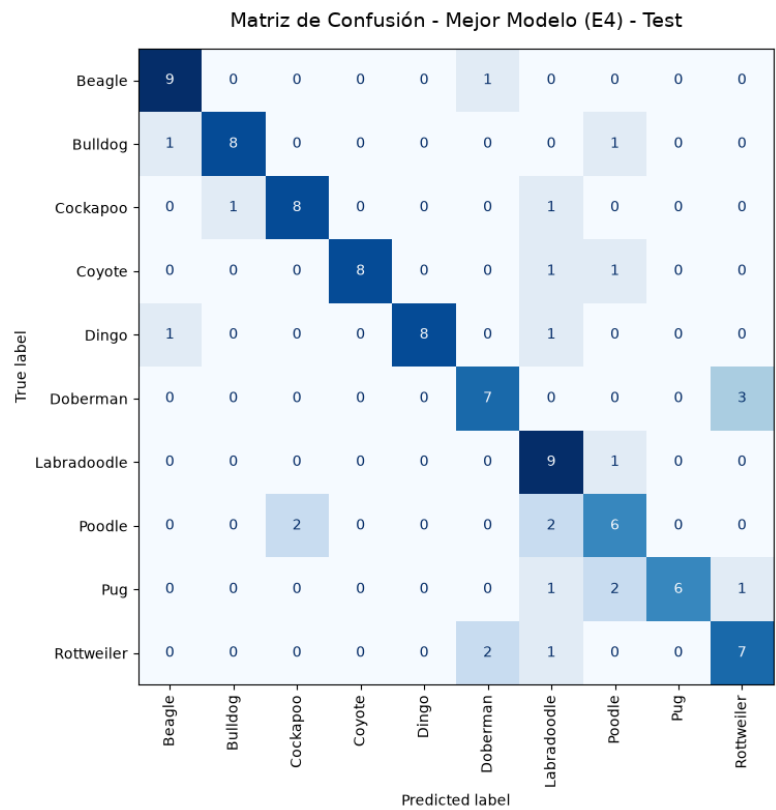


Imagen 11. Matriz de test de E4. Suma 76 aciertos sobre 100 imágenes. Doberman y Rottweiler acumulan cinco confusiones entre sí.

Ejemplos de Clasificaciones Correctas

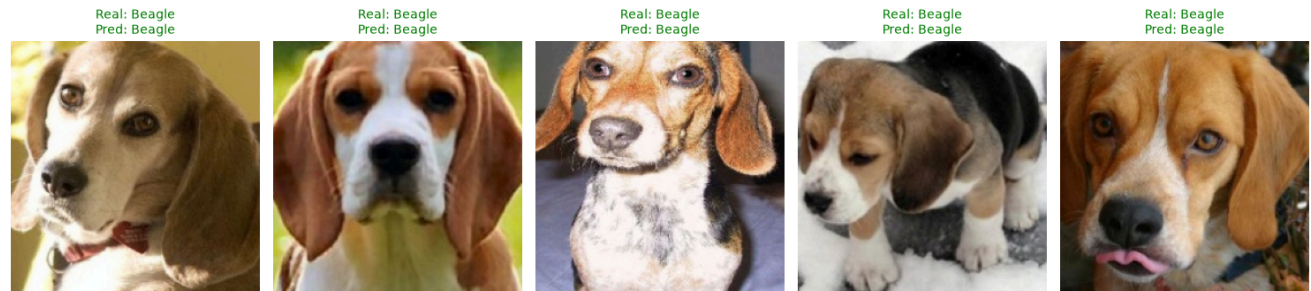


Imagen 12. Cinco predicciones correctas de E4 con sus etiquetas reales y predichas.

Ejemplos de Clasificaciones Incorrectas

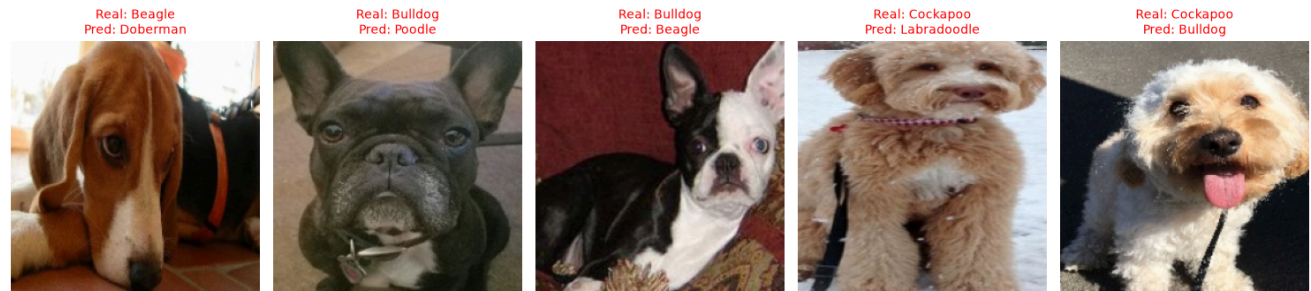


Imagen 13. Cinco predicciones incorrectas de E4. Las semejanzas de pelaje y los encuadres son hipótesis visuales sobre los errores.